淘宝AI购物助手在线评测落地实践

原创 营销&交易技术 大淘宝技术 Oct 11, 2026, 7:59 PM 浙江 8 listened



LLM-as-a-Judge 的学术验证已较充分，但在日均百万条会话、多域异构的线上 Agent 场景中全量落地，公开讨论还很少。本文以电商购物助手购后各子域为对象，完整记录了如何让 LLM-as-a-Judge 做到日级工业化运行：从线上脏数据的四层清洗、分布式推理架构、CNT 六层漏斗 Rubric 框架、Prompt 幻觉压制与跨轮隔离，到评测归因驱动业务优化的闭环。实践表明，评测可信度的瓶颈不在 Judge 模型本身，而在数据质量和评测标准的持续校准。经过多轮打标迭代，各域评测通过率收敛到较高且稳定的水平，评测归因已能区分数据层、Prompt 层、Rubric 层和 Agent 真实质量问题，进而驱动定点修复验证。

01 业务背景与问题定义

1.1 购物助手回复效果难评估，技术链路优化方向缺依据

购物助手是电商平台的 AI Agent 产品，**用户通过自然语言对话，完成问参数、查物流、办退款等购物服务**。业务侧目前分为购前和购后两大场景，本次重点场景是购后，覆盖用户下单后的全链路服务，涉及多个业务域，每天处理大量用户 query。

每条 query 从用户发出到拿到回复，中间经过路由决策 → skill 执行 → 工具调用 → 回复生成，涉及主框架、意图识别模型、工具接口、卡片渲染等多个上下游系统。任何一个环节出问题，用户拿到的回复就可能不对。

离线评测由测试团队在标准化用例平台上维护用例集，发版前做回归，设计为上线门禁。它测的是构造好的数据：query 精心设计、工具返回完整可控。线上不是这样的，用户表达模糊甚至带情绪，工具返回可能缺失或异常，数据分布持续漂移，因此离线通过率达标不等同于线上体验的完善。另一方面，AB 实验看转化率、留存这些业务指标，粒度太粗，即便转化率小幅下跌，我们也没法知道是某个域的路由出了问题还是某个域回复编造了信息，也就没法指导业务技术该优先优化哪个环节。

结果就是：回复效果没有持续的量化观测，出了问题靠人工捞 case 排查，技术链路的优化方向缺少数据支撑。

1.2 定义执行成功率，用日级自动评测替代人工抽检

为了看清复杂链路的细节，我们定义了**执行成功率：对每条线上 query，判定它是否被正确路由到对应 skill，并且被合理回应。**

它是一个三层漏斗的复合指标：路由准确率 × 回复正确率 = 执行成功率。前两层分别看路由和回复，执行成功率是端到端的综合判定，任何一层出问题都会拉低这个数字。

评测方式是基于多轮会话的 LLM-as-a-Judge：对每轮对话，自动判定用户诉求是否被有效推进。判定口径按业务场景拆分，不同域各有侧重（如时效准确性、流程合规、知识准确性），不存在用通用标准评估所有场景。

目标是把 Agent 质量观测从月度/季度人工抽检变成日级自动观测，形成"对话采集 → 任务判定 → 指标量化 → 口径迭代"的正循环。有了这个数据底层，技术团队就能看到哪个域通过率低、低在哪一环节（路由/工具调用/回复内容），从而为优化方向提供依据。

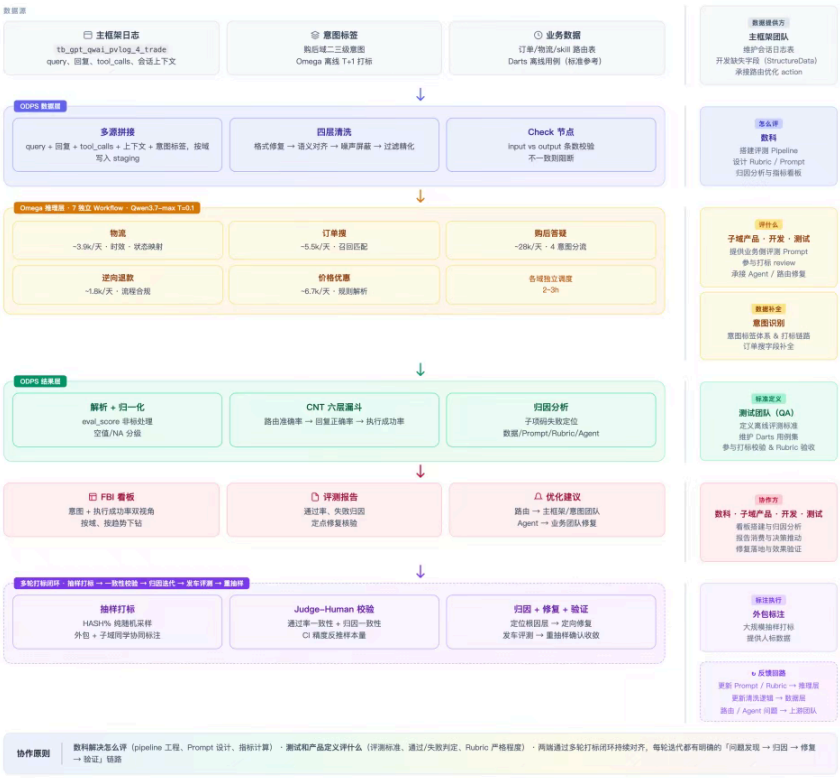
1.3 LLM-as-a-Judge 的学术验证较多，但全量落地的工业实践少

LLM-as-a-Judge 的方法论近两年已经历系统性的学术验证。Zheng et al. 的 MT-Bench 和 Chatbot Arena 是该方向的奠基工作，证明 GPT-4 做 Judge 与人类偏好一致性超过 80%，同时揭示了位置偏差、冗长偏差和自我增强偏差。后续工作主要两个优化方向：**评测质量上**，AlpacaEval的 length-controlled 版本用回归控制长度偏差，将与人类排名的相关性从 0.94 提到 0.98；Arena-Hard从众包数据中自动筛选高难度 prompt，以 500 道题达到 MT-Bench 三倍的区分度。**Judge 模型训练**上，JudgeLM证明 SFT 可将 GPT-4 的判定能力蒸馏到 7B 模型，一致性超过 90%，推理成本降一到两个数量级。

工业侧也有落地尝试。美团技术团队在 Agent 分层评测方案做了分层设计（系统级 → 任务级 → 轮次级），强调将模糊指标拆解为二元化规则来对齐评判标准。

但这些工作有共同特点：学术侧在受控环境评测，数据预先构造，不存在线上脏数据和分布漂移；工业侧多在千条级别验证，对日均海量会话、多域异构的场景，如何让 LLM-as-Judge 日级全量运行，公开讨论还很少。我们面对的就是这个场景：全量推理成本不可接受，不全量又保证不了覆盖率。本文要回答的是，怎么在保住可信度的前提下，让 LLM-as-Judge 做到工业化可持续运行。

整体方案分为五个阶段：**数据采集 → 数据清洗 → LLM 推理 → 结果解析与聚合 → 人工校验**闭环。端到端 pipeline 架构和团队协作关系如下图：



说明：

- 数据层和结果层用数仓 SQL 实现，推理层用推理平台 workflow，两端解耦。因此改 Prompt 不需要改数据逻辑，改过滤规则不需要重跑推理。各域各自独立 workflow，数据量级从数千到数万条/天不等，推理耗时均在数小时级别。各层技术细节详见第三部分。
- 评测不是一次性交付，而是持续收敛的过程。每轮迭代走「抽样打标 → Judge–Human 一致性校验 → 归因分析 → 优化 Prompt/Rubric → 发车评测 → 重抽样」的闭环，数科负责"怎么评"（pipeline 工程、Prompt 设计、指标计算），测试和产品定义"评什么"（评测标准、Rubric 严格程度），两端通过打标闭环持续对齐。

03 关键问题与解法

3.1 数据清洗与加工：从"原始日志"到"Judge 可消费的输入"

- 面对的问题

LLM-as-a-Judge 的评测质量不只取决于模型和 Prompt，更取决于喂给 Judge 的数据质量。线上 Agent 的原始交互日志不是为评测设计的（对比离线评测来自规范化测试数据，字段完整、格式统一），直接用来做 Judge 会导致模型不稳定、一致率低。具体问题：

- **数据源碎片化**：Agent 交互涉及用户 query、Agent 回复、工具调用链路、卡片渲染内容等多路信号，分散在不同的数仓表中，字段命名和格式各异。
- **评测数据缺失**：线上主框架日志没有落卡片的结构化内容，物流卡片、订单卡片等关键评测素材在线上评测时不可见；部分域的下单时间、商品标题、订单金额等信息也没有落到日志里。
- **tool_calls 中的噪声**：tool_calls_json 中包含工具接口的内部提示词（系统指令、开发注释等），这些不是 Agent 生成的内容，但会被 Judge 误读为 Agent 回复，导致评测偏判。
- **语义无法对齐**：同一概念在用户视角和系统视角下表达不同（如用户说"已揽收"但系统状态是某个英文枚举值，两者并不等于"已发货"），Judge 如果不理解这种映射关系，就会基于字面意思做出错误判定。

学术界和大多数工程实践通常假设输入已经是干净的、结构化的对话文本，数据准备被一笔带过。业界评测框架（如 AlpacaEval、MT-Bench）的输入数据来自受控实验环境，不存在线上生产系统的脏数据问题。少数涉及线上评测的实践（如美团 Agent 评测、EvEye）会做基础的数据清洗（去重、过滤无效会话），但通常不会深入讨论数据加工的细粒度策略和离在线数据差异的系统性处理。

- 我们的做法

面对上述现状，**我们建立了一套面向在线Judge 场景的四层数据清洗pipeline**，逐层解决数据质量问题。该pipeline并非一次性设计完成，是随着各域实际评测的推进依次暴露、定位、修复、沉淀得来。



- **第一层：修复数据格式不一致**。离线数据格式规范、字段完整；线上数据来自真实用户会话日志，各 skill 卡片协议独立、字段解析方式各异。直接复用离线 Prompt 会导致线上数据大量解析为空、评测结果不可用。因此在线评测对标离线 Prompt 的数据定义，逐域修复解析逻辑、补全字段映射。典型案例是某个域的卡片文本经常长达数千字，拼接后上下文超长导致推理超时甚至评测结果为空，需做合理截断。
- **第二层：语义对齐**。用户表达的订单状态与系统记录的状态存在语义差异，如用户说"已经揽收了"，系统状态字段是某个英文枚举值，但这两个表述对 Judge 来说并不天然等价，更不等于"已发货"。如果不做映射，Judge 会基于字面意思做出错误判定（如认为

Agent 说"已发货"与系统状态"已揽收"矛盾，判为回复错误）。因此在 Prompt 中显式建立用户语义与系统语义的映射。

- **第三层：屏蔽干扰信号**。tool_calls_json 中包含工具接口的内部生产链路提示词，这些不是 Agent 生成的内容但会被 Judge 误读为 Agent 回复。修复方式是用正则剔除工具接口中透传的生产链路提示词，只保留 Agent 实际调用工具的参数和返回值。这一步对所有域生效，屏蔽后各域评测准确率均有明显提升。
- **第四层：过滤规则精化**。早期各域的 query 入口过滤规则过于宽泛，大量不属于该 skill 的 query 被纳入评测。最典型的是某个流程类域，早期采集到的数据里大部分其实并非该域的真实意图（如用户只是咨询相关政策、并未真正发起对应操作，甚至完全不相关的 query），评测通过率因此极低且不可信；另一个知识问答类域也存在类似问题。修复方式是在各域评测入口增加基于 skill 标识的精确过滤。过滤精化后各域评测数据才真正可信，早期那个流程类域执行成功率异常偏低的问题，也在此步暴露出真实原因是 Prompt 不适配线上数据，而非线上 Agent 质量差。

3.2 平台使用与工作流设计：分布式推理的工程实践

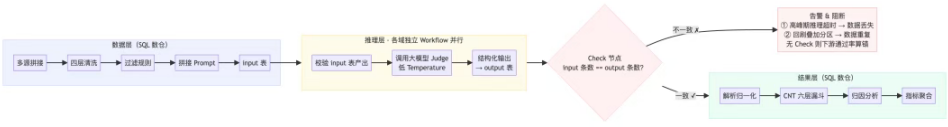
- 面对的问题

日级全量线上评测意味着每天要对海量会话逐条做 LLM 推理判定，这是一个分布式批量推理的工程问题：

- **规模与时效**：需要每天定时完成全量评测，输出当日执行成功率指标。单域 workflow 运行数小时，多域并行需要在次日凌晨前全部完成。
- **多域异构**：各评测域的 Prompt 模板、数据量级、推理耗时各不相同，不能用一套 workflow 一刀切。
- **稳定性要求**：日级调度意味着任何一次失败都会导致当天指标缺失，需要完善的容错和重试机制。

业界 LLM-as-Judge 评测在工程实现上大致有两种路径：一是直接调 API 做串行或小批量并行推理，适用于千条级别的小规模验证；二是借助 LangChain、LangSmith 等编排框架做批量调用，解决了并发控制和重试问题。这两种路径在小规模实验中够用，但面对日均海量、需要每日定时产出的线上评测场景，都面临同样的问题：推理任务和数据任务没有明确分工，出了问题不知道是数据错了还是模型推理错了，迭代时改一处就要重跑整条链路。

- 我们的做法



- **整体架构：数仓+推理平台串联**

我们没有把所有逻辑都塞进推理 workflow，而是按"数据归数据、推理归推理"的原则做了明确分工：推理平台只负责 LLM 推理任务（拼接 Prompt → 调用模型 → 返回结构化结果），前期的数据清洗、多源拼接、过滤规则，以及后期的结果解析、标签归一化、指标聚合，全部在数仓 SQL 中完成。这样做的好处是：

  - 推理平台保持轻量，不承载业务逻辑
  - 数据预处理和后处理都在数仓中用 SQL 实现，可复用、可回溯、可独立调试
  - 两端解耦后，改 Prompt 不需要改数据逻辑，改过滤规则不需要重跑推理，迭代效率更高。

- **数据完整性校验：Check 节点**

在推理完成后，设置了 check 节点：对比 input 表的条数与推理输出的 output 表条数，只有两者一致才进入下一步的结果解析。这是因为我们在使用过程中发现：

  - 公共推理资源池高峰期偶发推理超时导致部分数据丢失
  - 回刷同分区数据时，结果表是叠加而非覆盖分区

如果没有 check 节点，下游直接计算会导致通过率算错。

- **工作流设计：按域拆分独立 Workflow**

最初考虑过"一个大 workflow 跑全域"，实践中发现三个问题：

  - 短板效应：最慢的域拖住所有域；
  - 故障扩散：单域失败阻塞整个workflow；
  - 迭代耦合：改某域的 Prompt 需要重跑所有域。

最终采用各域独立 workflow 的方案，每个 workflow 独立配置队列、并发度和重试策略。

- **规模实测（示意）**

各域每日全量线上会话评测，单域单分区量级从数千到数万条不等，单分区 LLM 运行时间均在2–3h。

- 核心工程踩坑

- **资源竞争与排队**：使用公共推理资源池，正常调度约数小时完成，但大规模回刷或高峰期资源阈值触发排队，耗时可能成倍延长。解法是错峰调度 + 关键域申请独占资源。

- **Prompt 过长导致推理丢失**：部分域的 Prompt 拼接后 token 数较长（尤其是多轮对话 + 完整 tool_calls 的场景），超过模型上下文窗口后平台不会报错，而是静默返回空结果，导致该条数据评测缺失。需要在数据准备阶段做 token 数预检，超长时做截断或摘要处理。
- **回刷是叠加分区而非覆写**：推理输出到数仓分区时采用 append 模式。如果回刷历史分区前没有提前清空目标分区，新旧数据会叠加在一起，导致指标计算重复、通过率失真。回刷操作必须先清空目标分区再执行推理任务。

3.3 模型选型：Judge模型的选择策略

- 面对的问题

LLM-as-Judge 的底座模型选择直接决定评测质量的上限和成本的下限。选太弱的模型，评测准确率不达标；选太强的模型，日均海量推理的成本不可接受。更关键的是，不同模型在"遵循 Rubric 做判定"这个特定任务上的表现差异，可能远大于它们在通用对话能力上的差异。

学术界 LLM-as-Judge 研究中，主流做法是使用 GPT-4 级别的顶级模型作为 Judge，追求判定准确率最大化而较少考虑成本。工业实践中，部分团队尝试用中等模型做 Judge，在特定任务上可以达到接近大模型的准确率，但跨场景泛化性较差。近两年的研究趋势是用 SFT 微调小模型替代通用大模型做 Judge（如 JudgeLM、Auto-J），在标注充足的情况下小模型可以逼近大模型的判定一致性，同时推理成本下降一个数量级，但这要求有足够的标注数据。

- 我们的做法

- **当前阶段选择一款指令遵循能力强的大模型作为 Judge，选择逻辑如下：**
  - 任务特性匹配：Judge 任务本质是"基于 Rubric 的分类/判定"，给定会话上下文和评测标准，输出通过/不通过 + 理由。这不需要模型的创造性生成能力，而需要指令遵循能力和逻辑推理能力。所选模型在指令遵循和结构化输出方面表现优秀，且支持较长上下文。
  - 效果天花板验证：大模型 + 精心设计的 Prompt 组合下，各域通过率经过多轮迭代后收敛到较高水平，人工校验一致性达到可接受水平。
  - 成本可接受：在当前评测量级下，日均推理成本处于可承受区间。

- **分阶段的模型策略：**

- 【已完成】第一阶段：用大模型 + Prompt 工程快速验证效果天花板，确认"好 Prompt + 大模型"能达到的准确率上限。这一步的产出不仅是评测结果本身，更是后续 SFT 微调的对齐目标和标注数据来源。
- 【规划中】第二阶段：基于多轮打标沉淀的人标样本构建 SFT 训练集，微调垂直小模型替代大底座。团队内已有成功先例，某方向将大模型 SFT 蒸馏到小模型后，端到端 RT 明显下降、目标判定准确率大幅提升至接近大模型水平，证明小模型在特定判定任务上可以逼近大模型。

- **Temperature 调优：**

Judge 任务要求输出稳定可复现。我们对 temperature 做了梯度实验，最终定为 0.1：足够低以保证同一输入多次推理的判定一致性，又保留极小的随机性避免模型陷入固定模式。temperature 过高会导致同一 case 多次评测结果不一致，temperature 为 0 则偶发模型输出僵死。

- **选型认知迭代：**

早期我们尝试过更轻量的模型做 Judge，发现在以下场景表现明显不如大模型：

- 多条件复合判定：如某流程类域需要同时检查路由正确性 + 工具调用参数匹配 + 回复话术合规
- 边界 case 判定：如用户 query 模糊、Agent 回复部分正确
- 跨轮一致性判定：需要理解多轮对话的整体逻辑

这让我们认识到，Judge 任务的难度不在于"语言理解"，而在于"按照规则做复杂判断"，后者对模型的指令遵循和推理能力要求更高。

3.4 Rubric 设计：用户型产品的评测标准体系

- 面对的问题

如果 Rubric 本身定义不清晰、不统一，Judge 的输出就不可能可信。在购物助手这个场景下，Rubric 设计面临三重挑战：

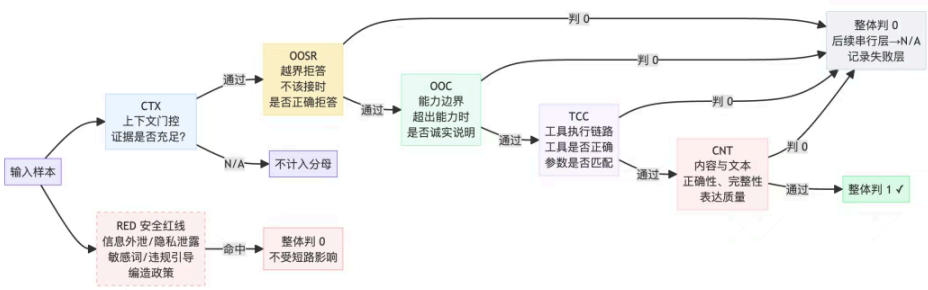
- **跨域标准不统一**：初期各域的评测规则由不同团队各自定义，各子域独立定义门禁条件和评测维度，有的域在信息不足时仍被判"不通过"，有的域直接跳过评测；评测项的粒度和口径也各不相同。这导致跨域通过率不可比、不可信。各域的内容评估本身肯定不一样（有的看时效、有的看流程），但评测的漏斗结构应该统一（即现在的 CTX/OOSR/OOC/TCC/CNT/RED 六大漏斗标准）。这是归因的基石。
- **用户型产品 + 长尾 query 的评测标准难定义**：购物助手是面向 C 端用户的产品，面向用户型产品的 Rubric 需要同时处理"标准模糊"和"场景多样"。不同于离线评测中确定

性测试用例，线上用户 query 天然模糊、多变、带情绪，Agent 的"正确回应"标准更难定义。同时线上数据呈现明显的长尾分布：头部模板 query（如"帮我退款"、"查一下物流"）占了大量会话，可以用精细化标准去评；而长尾 query（如用户在某个场景中突然问完全不相关的问题）则只能给宽容的底线规则。

学术界 LLM-as-Judge 的评测标准通常是二分类（正确/错误）或简单的 Likert 量表（1-5 分），适用于通用对话质量评估。工业实践中，美团 Agent 评测提出了分层评测框架（系统级 → 任务级 → 轮次级），每层定义不同的评测指标；EvEye 则强调"端到端成功率"作为北极星指标。但大多数实践中，Rubric 如何跨域统一、如何适配用户型产品的模糊性和长尾性，较少被深入讨论。

- 我们的做法

1. CNT 六层漏斗框架统一



我们对全域评测标准做了系统性重构，建立了统一的 CNT 六层漏斗框架。当前所有评测域全部迁移到同一框架下，每层的评测功能和取值口径全局统一：

- **CTX**（上下文门控）：判断样本是否具备评测所需证据。这是唯一产生 N/A 的层，证据不足时整条样本标记为不可评、不进入通过率分母。设计核心是"诚实跳过"优于"强行评分"：宁可剔除一条不确定的样本，也不让 Judge 在信息缺失时猜一个 0 或 1。
- **OOSR**（越界拒答）：当 query 不该由当前 skill 承接时，Agent 是否正确拒答或交回上层。
- **OOC**（能力边界诚实）：当 query 在 skill 范围内但超出当前能力时（如某检索类域不支持某种筛选条件），Agent 是否诚实说明边界而非编造回答。
- **TCC**（工具执行链路）：Agent 是否调用了正确的工具、参数是否匹配意图、调用链路是否自洽。
- **CNT**（内容与文本）：Agent 回复的内容正确性、完整性和表达质量。这是最接近用户感知的层，按域差异最大。
- **RED**（安全红线）：内部信息外泄、隐私泄露、敏感词、违规引导、编造政策五条红线。RED 与串行层并联、恒评，即使前面的层判了 0，RED 仍然独立核查。

六层之间用短路机制串联：任一层判 0 → 后续串行层记 N/A、整体判 0（短板原则），但 RED 不受短路影响。这个设计让评测结果同时具备"主因定位"（失败发生在哪一层）和"安全兜底"（RED 独立覆盖）两个维度。

2. 统一框架 + 分域设置特异性子项

框架统一不等于 Rubric 统一。每个域在六层框架下有独立的子项码体系，各域用各自的前缀标识，子项数量在十几个上下浮动。差异主要来自两个方面：

- CNT 层的域特异性：这是各域差异最大的层。比如时效类域的 CNT 子项关注时效描述是否凭空编造（如凭空声称"已签收"）；检索类域的 CNT 子项需要逐条校验时间 / 商品 / 属性 / 状态 / 店铺 / 逻辑组合等多个维度的召回匹配；服务规则类域的 CNT 子项则关注服务规则是否被反向弱化或编造。
- 本地扩展处理域特有场景：有的域新增了"空态硬退出"子项——当查无结果且系统如实返回空态时直接判通过，因为"查不到"本身就是正确答案。这些扩展不破坏框架统一性，而是在 CTX 层内增加了域特有的"通过态硬退出"分支。

3. 面向用户型产品的 Rubric 特性

购物助手是 C 端产品，Rubric 设计必须适配线上用户行为的特殊性，这和离线评测中确定性测试用例有本质区别。我们在 Rubric 中做了三项针对性设计：

- **模式判定与诚实兜底放行**：同一个 query 在工具返回有数据（模式 A）和无数据（模式 B）时，Agent 的"正确回答"完全不同：模式 A 应该给出具体规则，模式 B 忠实兜底 + 引导查看详情就是产品预期行为。Rubric 中显式定义了模式判定规则（基于返回数据是否为空），模式 B 忠实兜底直接判通过，不因"缺具体规则说明"而扣分。这在离线评测中不存在，离线用例总是构造好完整数据的。
- **意图码驱动的评测路径分流**：某个知识问答类域下有多个二级意图，每个意图的关注要素不同，Rubric 通过意图码变量驱动评测路径分流。更特殊的是其中某一类意图，命中即 CTX 硬退出判通过，因为这类意图代表"正向场景"，无需进入常规评测链路。
- **卡片结构化数据盲区兜底：**线上主框架不返回卡片的结构化内容，但部分评测逻辑本应依赖卡片数据。Rubric 中的处理方式是：将该子项留空、恒判 N/A，并在取证总原则中显式声明"严禁假设可读取卡片结构化数据"、"严禁因为看不到该数据而判 0"。这不是理想的工程方案，但在当前数据限制下是最务实的选择。

4. Rubric 的版本化管理与代码化存储

每个域的 Rubric 以 Markdown 文件存储在代码仓库中，包含完整的 YAML 版本元数据和变更日志。以某个域为例，其 Rubric 经历了从初版到当前版本的数十次迭代，每次变更都有明确的 changelog（日期、版本号、具体改了什么、判分影响预估）。这样 Rubric 的变更可以像代码一样 review 和追溯，测试同学可以快速迭代验证新标准效果，离线和在线的

Rubric 快照保持同步，在线评测的 Prompt 直接从 Rubric 文件生成，不存在"文档写一套、Prompt 写另一套"的问题。

3.5 Prompt 设计：让 Judge 输出稳定可信

- 面对的问题

即使 Rubric 定义完善、数据清洗到位，Prompt 的细节设计仍然直接决定 Judge 输出的稳定性和可信度。LLM 对 Prompt 的敏感度远超预期，在实践中，同样的评测标准，Prompt 写法不同可能导致通过率明显波动。核心挑战在于：Judge 需要严格按照 Rubric 做判定，而不是"自由发挥"。但 LLM 天然倾向于生成流畅的文本而非精确的判定，容易产生以下问题：

- **幻觉**：模型"脑补"用户意图或工具行为，对不存在的证据做出判定。
- **格式不稳定**：模型不按指定的 JSON schema 输出，评分字段出现非标准值，导致下游解析失败。
- **时间判断局限性**：Agent 回复中的时间相关表述（"明天到达"、"3 个工作日到账"），Judge 无法判断准确性。
- **维度间串味**：多维度评测时，模型把 S1 的判定理由混入 S2 的输出字段。

学术界 LLM-as-a-Judge 的 Prompt 设计通常比较简洁——给一段系统指令 + 输入数据 + 期望输出格式。MT-Bench 使用 chain-of-thought 引导模型先思考再打分。AlpacaEval 使用 reference-guided 方式，让 Judge 参考标准答案做对比评判。工业实践中，常见做法是在 Prompt 中加入 few-shot 示例来引导输出格式。但针对 Judge 场景的系统性 Prompt 工程（幻觉压制、时间锚点、干扰屏蔽等）较少有深入讨论。

- 我们的做法

1. 输入输出变量声明与结构化定义

在 Prompt 开头明确声明所有输入变量的含义和格式（如 user_query、agent_reply、tool_calls_json、session_context 等），避免模型对字段含义产生歧义。对输出格式做严格约束：明确 JSON schema，指定每个字段的类型、枚举值和必填/可选。例如评分字段只允许输出 0/1/剔除，评分理由字段必须为字符串。对于多维度评测（S1~S6），逐维度定义判定标准和输出位置，避免模型"串维度"打分。

2. 压制模型幻觉、提升判定稳定性

这是 Prompt 设计中最关键也最容易被忽视的部分。我们沉淀了四项实践：

- **锚定事实来源**：在 Prompt 中显式要求"仅基于以下输入信息进行判断，不得引入外部知识或假设未提供的上下文"。Judge 场景下模型容易"脑补"用户意图（如把用户的模糊 query 自行补全为明确意图），锚定约束能有效减少此类幻觉。
- **反向验证指令**：要求模型在给出判定前，先引用输入中的具体文本作为证据（"请先摘录 Agent 回复中与用户 query 直接相关的原文片段，再据此判断"），强制模型 grounded 在输入事实上。
- **边界 case 的 Few-shot 示例**：针对高频误判场景（如用户 query 模糊、Agent 回复部分正确），提供 2–3 组边界 case 的标注示例，让模型理解"什么程度算通过、什么程度算失败"的判定边界。
- **禁止推断性判断**：明确声明"若输入信息不足以做出判断，请输出'剔除'而非猜测性打分"，避免模型在信息缺失时强行给出 0/1 判定。

3. 提问时间拼接——解决时间敏感场景的误判

Agent 回复中经常包含时间相关信息（"您的订单预计明天到达"、"退款将在 3 个工作日内到账"），用户提问也经常包含时间信息（如：帮我查近一年订单共花了多少钱），但 Judge 模型没有当前时间概念，无法判断"明天"、"近一年"是否准确。

解法：在输入中拼接用户提问的精确时间戳，并在 Prompt 中说明"请基于该时间戳作为参考基准来评估 Agent 回复中涉及时间表述的准确性"。

效果：修复了时效类域的"时效篡改"类问题（如 Agent 回复"预计后天到"但实际已过时效），也解决了检索类域因时间锚点异常导致的大量误判。

4. 分域 Prompt 适配而非通用 Prompt

不同域的评测重点不同：时效类域重时效准确性和状态描述、流程类域重流程合规和参数匹配、知识问答类域重知识准确性和回答完整性。通用 Prompt 无法覆盖各域特殊性，需按域定制评测维度和判定细则。例如某个流程类域按 query 意图（提交申请 / 确认 / 撤销）拆分子判定标准，而非对该域所有 query 做统一要求。

5. 跨轮信息隔离——上下文不是越多越好

多轮对话场景下，我们原本做法是把完整的对话历史和全部工具调用链都传给 Judge，让它拥有最充分的上下文。但某个流程类域的实践证明了一个反直觉的结论：给 Judge 更多上下文不一定更好。

最初我们把全对话的工具调用链传给 Judge，结果模型会用前一轮的工具返回来评判当前轮的 Agent 回复（如前一轮返回的某个金额被 Judge 当作当前轮的事实依据），导致跨轮信息互相干扰串味，通过率明显偏低。改为只传当条调用的 tool_calls 后，通过率显著跃升。

原因在于LLM 的注意力分配机制：当上下文中存在多个轮次的工具调用信息时，模型难以区分"当前轮该看哪个"，容易被相邻轮次的信息干扰。Prompt 设计中需要根据这个特性做精准的信息投喂，而不是无差别地堆砌上下文。

6. 解析侧兜底——结构化输出的三层防线

Prompt 设计再好也无法 100% 杜绝非标输出。因此解析侧还有三层兜底：

- Prompt 约束：强制模型输出结构化格式（各维度严格输出 0/1/剔除）。
- 输出校验器：解析层拦截非标输出，标记为解析失败。
- 标签归一化：对评分字段做归一化处理（新旧格式混合、输出空值/非标准值的分级 CASE WHEN 处理）。

踩坑：数仓正则提取函数存在转义陷阱，SQL 中的反斜杠会被逐层解析，导致长文本提取的正则表达式需要反复调试才能稳定工作。

3.6 多轮打标优化：逼近"真实线上水位"的迭代闭环

- 面对的问题

LLM-as-a-Judge 的评测结果最终需要回答一个问题："这个通过率数字可信吗？"回答这个问题的难点在于：我们不知道"真实的线上水位"是多少，没有一个 ground truth 可以对照。离线评测的通过率不能直接搬到线上（数据形态不同），人工抽检的通过率样本量有限且有标注偏差，AB 实验的业务指标又太粗无法对齐。因此只能通过多轮迭代，让评测结果逐步收敛到认知中的真实水位。

学术界验证 LLM-as-Judge 可信度的标准做法是计算 Judge-Human 一致性（Cohen's Kappa 或 Krippendorff's Alpha），通常报告 0.6-0.8 的一致性系数即认为可接受。工业实践中，多数团队在评测上线前做一轮人工标注校验，确认准确率达标后即投入使用。但这种"一次性校验"有两个问题：没有考虑评测标准本身可能需要调整（如果 Rubric 定义偏了，即使 Judge 和人标一致，结果也不可信）；也没有持续的校验机制，模型更新、业务变化后，原有的一致性可能不再成立。

- 我们的做法

我们建立了一套**「抽样打标 → 对比分析 → 优化 Prompt/Rubric → 发车评测 → 重抽样」**的多轮闭环迭代机制，联合各子域产品、开发、测试同学协同迭代：



迭代过程与效果实证（下表用中性类别词指代不同域，通过率只给趋势）：

业务域（中性代称）	通过率变化	核心优化动作
时效类域	稳步小幅提升	清洗 tool_calls 超长提示词；修复某意图识别逻辑
流程类域	显著跃升	tool_calls 从全对话工具链回退为当条调用（消除跨轮信息串扰）
知识问答类域	显著跃升	修改 Prompt + 新增意图码 + 数据源切换到更完整的路由信息表

- 时效类域的多轮迭代：初版通过率已处于中上水平，分析发现部分低分 case 是因为 tool_calls 中包含超长的系统提示词，Judge 误认为 Agent 输出了大量无关内容；清洗这些提示词后通过率进一步提升；再修复意图识别逻辑中的边界 case 后，通过率略有回落但更准确（少量原"通过"被修正为"不通过"）。
- 流程类域的多轮迭代：初版通过率远低于预期，深入分析发现根因是跨轮信息串扰——全对话工具链中前几轮的工具返回被 Judge 用来评判当前轮的 Agent 回复，导致误判（详细的注意力机制分析见 3.5 第 5 点）。改为只传当条调用的 tool_calls 后，通过率显著跃升。
- 知识问答类域的多轮迭代：最大的提升来自两个改动，一是新增意图码字段，让 Judge 知道当前 query 属于哪个二级意图，从而使用对应的判定标准；二是数据源切换到包含更完整路由信息的表。

Judge–Human 持续校验

每轮迭代后都进行 Judge 与外包人工抽检的一致性校验。抽样口径采用纯随机（HASH% 采样率），保证通过率无偏。CI 精度反推样本量：±1pp 需要约 4k 样本、±0.5pp 需要约 15k 样本。通过率波动若远超 CI 宽度，应排查 Rubric/模型等实质变化而非抽样噪声。

单点 Case 修复归因

当抽样打标或线上监控发现某条样本的评测结果存疑时，需要追溯这条 case 的问题根因到底在哪一层，然后在正确的层做修复。实践中，单条 case 的失败根因可以归为四类：

- **数据层问题**：输入给 Judge 的数据本身有问题。典型如 tool_calls 中未清洗的系统提示词被 Judge 误读、卡片超长截断导致关键信息丢失、字段映射错误导致语义偏差。修

复方式是调整数据清洗 SQL 或字段解析逻辑，影响面是该域所有同类数据。

- **Prompt 层问题**：数据是对的，但 Judge 对评测规则的理解有偏差。典型如某类边界 case 在 Prompt 中缺少判定示例，导致 Judge 按自己的理解"自由发挥"；或者子项码之间的判定边界描述模糊，Judge 把 A 子项的失败归因到 B 子项。修复方式是补充 Prompt 中的判定规则或 few-shot 示例，影响面是该域所有同类 case。
- **Rubric 层问题**：评测标准本身和技术认知不一致。典型如某个场景技术团队认为 Agent 的回复是正确的（如模式 B 忠实兜底），但 Rubric 中的规则会导致判 0。修复方式是修改 Rubric 定义并同步更新 Prompt，影响面最大，需要同步更新离线和在线的 Rubric 快照。
- **Agent 真实质量问题**：排除以上三类后剩余的失败 case，才是 Agent 本身的真实缺陷，回复确实编造了事实、工具调用确实传错了参数、确实触发了安全红线。这类 case 需要反馈给 Agent 开发团队做模型或逻辑修复。

早期没有归因机制时，看到通过率下降的第一反应是调 Prompt，结果有时越调越差，因为根因在数据层或 Rubric 层，改 Prompt 只是在掩盖问题。有了明确的四层归因后，每条存疑 case 都会先定位根因再决定修复层，修复效率和准确率显著提升。

多轮打标优化的目标不是"让通过率变高"，而是"让通过率更可信"。通过率的提升是因为评测标准逐步逼近了"真实线上水位"，早期低通过率不是线上 Agent 真的那么差，而是评测方法还没调对。当各域通过率收敛后，后续的变化才是真正有意义的，每一次下降都可能指向一个真实的线上质量问题。

04 评测归因：从"看水位"到"驱动优化"

4.1 评测的价值在于归因

执行成功率跑出来只是一个数字，数字本身不产生价值，价值在于拿到数字之后做了什么。

早期产出成功率数值后所有人听到后的第一反应是"那到底哪里该改"。因为评测如果只停留在报数，跟 AB 实验说"转化率跌了一点"没有本质区别，知道不好，但不知道改哪里。

因此我们从一开始就把归因作为评测的核心产出。经过迭代，最终归因分两个层次：**宏观上按 CNT 六层漏斗和子项码维度拆解失败分布**，告诉业务"低在哪一层、集中在哪些子项"；**微观上对具体问题构建 SQL 可复跑的检测规则**，在修复前后两个日期分区对比命中数，验证修复是否生效。

4.2 指标异动归因案例：某域路由成功率骤跌

某天起，某个域的路由通过率出现骤跌。直觉反应是 skill 能力回归，但归因后发现没有问题。逐日拆解分子（匹配到该 skill 的 trace）与分母（该域 base trace）发现：分子几天内保持平稳，分母却在两天内先翻倍、再接近三倍。

进一步核查发现，全站日 query 总量保持平稳，不是流量涌入，而是该域意图占比出现了异常抬升。按子 label 拆解后定位到：真实的几个业务动作全程平稳，暴涨集中在其中一个 label 上，量级近乎翻了一个数量级。这批 query 原文几乎全是使用类咨询（如"某功能异常怎么排查""某配件怎么处理"），根因是上游另一个 skill 放量后，意图模型把这类使用咨询 query 误标成了该域的投诉类 label，被该域 base 过滤条件抓进了分母。而这批暴涨的 trace 中，真正进过该域 skill 的占比极低。

剔除该 label 后，路由通过率五天的跌幅全部消失、恢复平稳。**不做归因就会以为线上有一个紧急 bug 需要修复，其实只要从评测过滤条件中去掉一个不属于该域真实动作的 label 即可。**

4.3 定点修复核验：通过评分理由字段判断问题收敛

某个时效类域经过三轮 Prompt 迭代后整体通过率趋于稳定，分析失败样本定位出五个子问题交由服务端修复。针对每个问题构建 评分理由字段 检测规则（SQL 可复跑的正则匹配），在修复前后做命中数对比，观察各类问题是否收敛：

问题类型	修复前后变化	判定
时效篡改·矛盾	小幅下降	部分好转
某类措辞问题	小幅上升	未收敛
状态边界/数量/时效/语言	大幅下降	大幅收敛
无兜底 + 张冠李戴	基本持平	占失败池比反升
卡片状态 vs 文案不统一	显著下降	显著修复

总失败样本明显下降，整体通过率稳步提升。5 类问题中 2 类显著修复、1 类大幅收敛、1 类部分好转、1 类未收敛。未收敛的那类措辞问题被标记为下一轮重点。

这个核验模式在各域复用：构建检测规则 → 修复前后分区对比 → 判断收敛 → 未收敛进入下一轮。虽然不是干净的 A/B 实验，但在工程实践中足够判断修复是否生效。

4.4 落地收益与当前边界

评测驱动优化的闭环已经跑通：评测发现问题 → 归因定位根因 → 排入排期表 → 技术/业务修复 → 评测验证收敛。各域评测指标也已接入看板日级监测：



说明：报表数字的校准主要来自 3.6 节的评测方法论实践，而非 Agent 本身能力提升。真正由评测驱动、业务侧完成修复并验证收敛的案例集中在定点问题层面（如 4.3 节的状态边界和卡片状态不统一问题）。

当前归因体系还停留在"发现问题 + 定位根因"阶段，问题修复仍依赖人工排期推动，评测坏样本尚未自动回流到 Agent 训练链路（SFT/GRPO）。

05 现存局限与未来规划

5.1 在线评测能力建设全景与当前进展

在线评测不是一个一次性的技术方案，而是一套需要日级运行、持续迭代、能反哺业务优化的能力体系。**这套体系的建设不止于"能打分"，还涉及评得准不准、评不评得起、评测结论怎么驱动修复、系统自身怎么持续运营**。我们沿五级成熟度（L1 能评 → L2 评得准 → L3 省又全 → L4 自动闭环 → L5 自运营）展开，将已有工作逐项落位：



实践下来，搭建一条能日级运行的评测链路，最不可或缺的是三块：指标定义（Rubric + CNT 框架）、Pipeline 工程化（数仓 + 推理平台自动化）、观测能力（BI 看板 + 日级异动归因）。精度校准在多轮打标中自然推进了一部分，但系统性偏差治理还没开始。当前日级运行已经稳定，下一步重点是降本和扩域：

- 【已完成】L1 能评：有 Rubric、有 Judge、能打分。执行成功率三层漏斗定义、CNT 六层 Rubric 框架、分域子项码体系、Rubric 版本化代码管理、离在线快照同步均已完成。在线指标与离线通过率、业务 KPI 的三方对齐验证尚未做，确定性规则与 Judge 判断的系统性剥离也未启动。
- 【进行中】L2 评得准：Judge-Human 一致性校验（CI 精度反推样本量）、幻觉压制四项实践（锚定/反证/Few-shot/禁止推断）、时间锚点拼接、跨轮信息隔离、结构化输出三层防线、Temperature 梯度实验已完成。位置偏差（交换双评）、冗长偏差（length-controlled）、宽松偏差（pairwise 替代）三类已知偏差的系统性治理尚未启动；多 Judge 投票和元评测（Judge 准确率做成上线监控）也未开始。
- 【部分自动】L3 省又全：大模型 + Prompt 验证效果天花板、错峰调度、各域全量日级评测、意图码驱动路径分流已完成。SFT 微调小 Judge 已规划未启动。采样科学化、级联 Judge、Batch API / Prompt Caching、另一大场景扩展、长尾难例挖掘与 Slice 分层均未使用。
- 【部分完成】L4 自动闭环：数仓 + 推理平台 Pipeline 全链路自动化、各域独立 Workflow 并行、四层归因 + 定点核验、多轮打标驱动 Prompt 迭代、指标异动归因均已完成。Release Gate（模型变更后自动评测、不达标阻断上线）未建设；坏样本到 SFT/GRPO 训练链路的自动回流未打通。
- 【部分完成】L5 自运营：BI 双视角看板、日级观测与异动归因、评测报告自动产出已完成。Drift 检测告警和 7×24 数字员工推送待做。

5.2 下一步规划

- **降本**：当前大模型 Judge 日均推理量较大，月级成本压力不小，系统性降本是继续扩域和持续运行的前提。SFT 微调小 Judge 是主路径，团队内某方向已验证可行性：将大模型 SFT 蒸馏到小模型后，端到端 RT 明显下降、目标判定准确率大幅提升至接近大模型水平。采样科学化（在保证精度的前提下，所需样本量相比全量可大幅下降）、级联 Judge、Batch API / Prompt Caching 同步推进。覆盖侧同步启动另一大场景的评测链路搭建。
- **补精度地基**：位置偏差（交换双评消除）、冗长偏差（length-controlled 校正）、宽松偏差（pairwise 替代绝对判定）三类偏差的系统性治理；多 Judge 投票或分维度小 Judge；元评测把 Judge 准确率做成上线监控指标。精度是评测结论可信度的保障，降本之后如果精度不够，降的是错误结论的产出效率。

- **闭环接通后训练**：打通评测坏样本到 SFT/GRPO 训练链路的自动回流，建设 Release Gate 与 Drift 触发深评机制，实现"评测发现问题 → 训练修复问题 → 评测验证修复"的自动循环。同期推进用户隐式反馈融合（跳出率、差评、负反馈与执行成功率联合校准）和评测 A/B 实验平台（Rubric、Judge 模型灰度对比）。

优先级判断：成本是入场券，精度是护城河，闭环是复利。数科的不可替代性集中在指标标准设计、精度校准和数据闭环三块，它们的核心是掌握业务标准与数据，而不是训练技术本身。



本文作者盈衣，来自淘天集团-交易数据技术团队。该团队致力于面向淘宝交易履约链路（详情、下单、支付、购物车、订单、物流、逆向、评价等）海量数据挖掘DAU、DAC及用户体验增长机会。同时，团队持续探索AI数据智能（实验归因、异动归因、智能取数）、AI-Native数据研发模式、AI Agent产品分析、评测与训练方向，推动Agent产品策略优化与迭代飞轮。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术
服务端技术 | 技术质量 | 数据算法

大淘宝技术
大淘宝技术官方账号
965篇原创内容
公众号

阅读 6391

留言

写留言



还没有留言，来写首评